

# Bienvenue sur AWS

---

## Compte, budget, IAM et vos premières commandes CLI

CL-AWS1 — Jour 1 — Votre premier jour sur un vrai cloud

---

### Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- **Expliquer** ce qu'est un compte AWS (frontière de facturation **et** de sécurité), une région, une zone de disponibilité.
- **Poser un garde-fou financier** : alerte de budget à 10 USD avec seuils 50/80/100 % — *avant* de créer quoi que ce soit.
- **Sécuriser le compte** : MFA sur le root, utilisateur IAM du quotidien, groupes, politiques.
- **Lire une politique IAM en JSON** : Effect, Action, Resource, Condition — et prédire qui peut faire quoi.
- **Piloter AWS depuis votre terminal** avec la CLI v2 : profils, `--query`, `--output`.

Fil conducteur du cursus, toujours le même : **automatiser, superviser, sécuriser**. Aujourd'hui, on pose le socle des trois.

---

### Plan de la journée

1. **Découverte d'AWS** — le compte, la console, les régions et zones de disponibilité, la facturation à l'usage et le free-tier.
2. **Le budget d'abord** — l'alerte AWS Budgets avant toute ressource.  Démo 1.1.
3. **IAM** — root vs utilisateurs, groupes, anatomie d'une politique JSON, MFA, bonnes pratiques.  Exercice 1.1.
4. **AWS CLI v2** — installation, `aws configure`, profils, premières commandes, `--query` et `--output`.  Démo 1.2,  Exercice 1.2.
5. « **Tout est API** » — console, CLI, SDK : trois clients d'une même chose.

Puis : récap, quiz de fin de journée, et le teaser du Jour 2.

---

## 1. Découverte d'AWS

---

### Le compte, la console, les régions — vos nouveaux repères

---

#### Votre compte AWS : une double frontière

Chacun d'entre vous dispose d'un **compte AWS sandbox individuel** (déjà créé et vérifié). Un compte AWS, c'est deux choses à la fois :

- **Une frontière de facturation** : tout ce qui est créé *dans* le compte est facturé *au* compte. Une seule facture, un seul payeur. Personne d'autre ne paie pour vous, vous ne payez pour personne d'autre.
- **Une frontière de sécurité** : par défaut, **rien ni personne** en dehors du compte ne peut toucher à ce qu'il contient. Les identités (utilisateurs, rôles) vivent dans le compte.

Chaque compte a un **identifiant à 12 chiffres** (ex. **123456789012**) — vous le reverrez partout, notamment dans les ARN.

Analogie avec vos acquis : le compte AWS est à votre infrastructure ce que la **VM du TP1** était à StockLine — le périmètre dont vous êtes responsable. Sauf qu'ici, le compteur tourne.

---

## Le tour du propriétaire : la console

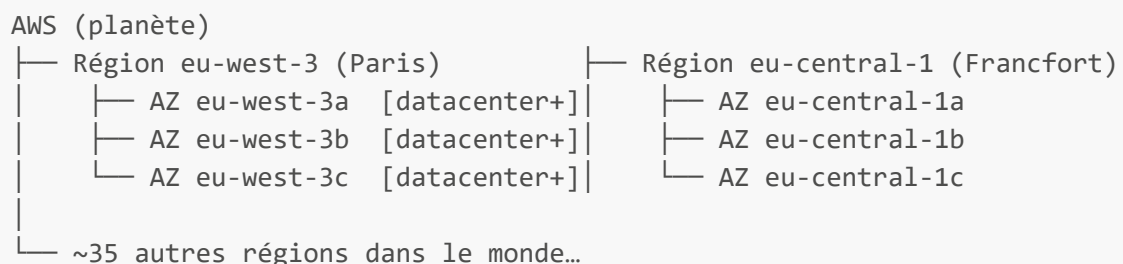
La **console AWS** (<https://console.aws.amazon.com>) est l'interface web. Vos repères :

- **Barre de recherche** (en haut) : le moyen le plus rapide d'atteindre un service — tapez « IAM », « Budgets », « EC2 ». Il y a **plus de 200 services**, personne ne navigue par menus.
- **Sélecteur de région** (en haut à droite) : indique *où* vous travaillez. Réflexe n°1 : **vérifier qu'il affiche « Paris » (eu-west-3)** en arrivant.
- **Menu du compte** (votre nom, en haut à droite) : facturation, paramètres du compte, déconnexion.
- **Services récemment visités** sur la page d'accueil.

⚠ La console **change régulièrement** d'apparence. Les captures d'écran vieillissent mal — les *concepts*, eux, sont stables. C'est aussi pour ça qu'on passera vite à la CLI.

---

## Régions et zones de disponibilité



Région = zone géographique, isolée des autres

AZ = un ou plusieurs datacenters, alimentation et réseau indépendants, reliés entre eux en fibre à très faible latence

Une **région** = un lieu du monde. Une **AZ** (Availability Zone) = un « bâtiment » (ou groupe de bâtiments) indépendant dans ce lieu. Une panne d'AZ ne doit pas emporter la région : c'est la base de la haute disponibilité.

---

## Une ressource vit dans SA région

Point capital, source de la panique classique du débutant :

- La plupart des ressources (machines virtuelles, réseaux, bases...) sont **régionales** : créées à Paris, elles **n'apparaissent pas** dans la console si le sélecteur est sur Francfort.
- « J'ai perdu ma VM ! » → non, vous êtes juste **dans la mauvaise région**. Vérifiez le sélecteur avant de paniquer.
- Quelques services sont **globaux** : IAM (les identités valent pour tout le compte), Route 53, CloudFront. Pour eux, le sélecteur affiche « Global ».

Conséquence facturation : une ressource oubliée dans une région où vous n'allez jamais **continue de coûter**. D'où notre garde-fou de tout à l'heure : une politique qui **interdit toute action hors eu-west-3**.

Analogie réseau (bloc 04) : les régions sont des **réseaux isolés** les uns des autres — rien ne « fuit » d'une région vers une autre par défaut.

---

## Pourquoi eu-west-3 (Paris) ?

Toute la formation se passe dans **eu-west-3**. Trois raisons, les mêmes qu'en entreprise :

- **Latence** : vos utilisateurs (et vous) êtes en France. Paris  $\approx$  quelques millisecondes ; un aller-retour vers **us-east-1** (Virginie)  $\approx$  80-100 ms. Pour une API comme StockLine, ça se sent.
- **RGPD / souveraineté** : les données restent **physiquement en France**. Pour beaucoup de clients (santé, secteur public, banque), c'est une exigence contractuelle, pas un confort.
- **Prix** : les tarifs varient **par région**. Paris est légèrement plus chère que la Virginie, mais le choix de région est d'abord dicté par la latence et la conformité — pas par 5 % d'écart de prix.

Réflexe professionnel : le choix de la région est une **décision d'architecture**, prise en premier, documentée, et rarement changée ensuite.

---

## Le modèle de facturation : paiement à l'usage

Changement mental majeur par rapport à votre VM du TP1 :

- **Pas d'achat, pas d'abonnement forfaitaire** : vous payez ce que vous consommez, à la seconde, au Go, à la requête, selon le service.
- Trois grandes familles de coût : le **calcul** (durée  $\times$  taille de la machine), le **stockage** (Go  $\times$  mois) et le **transfert de données** (surtout les données *sortantes* d'AWS).
- Une ressource **arrêtée** peut encore coûter (son disque, son IP publique...). Une ressource **supprimée** ne coûte plus rien.
- Le danger n'est pas le prix unitaire (souvent centimes/heure) : c'est **l'oubli**. Une machine oubliée 3 semaines, ça se voit sur la facture.

D'où la règle d'or du bloc : **teardown systématique** en fin de séance, et une alerte de budget posée avant tout.

---

## Le free-tier : votre filet (troué)

AWS offre un **niveau gratuit** (free-tier) sur les nouveaux comptes. Trois types d'offres :

- **12 mois gratuits** : ex. 750 h/mois d'instance **t2.micro/t3.micro** (assez pour UNE machine allumée en continu), 5 Go de stockage S3...
- **Toujours gratuit** : ex. 1 million de requêtes Lambda/mois, 25 Go DynamoDB, et... **les 2 premiers budgets AWS Budgets** — celui qu'on crée ce matin est gratuit.
- **Essais limités** : X jours gratuits sur certains services, puis facturation.

⚠ Le free-tier est un filet **troué** : il couvre *une t3.micro*, pas deux ; il ne couvre **pas** les NAT Gateway, load balancers, IP publiques inutilisées... On vous signalera chaque service payant par un encadré 💰.

💰 **Aujourd'hui : 0 €**. Budgets (2 premiers gratuits), IAM (gratuit), CLI (gratuit — c'est un logiciel sur *votre* machine). Tant qu'on reste là, rien ne peut être facturé.

---

## 2. Le budget d'abord

---

### Aucune ressource avant l'alarme incendie

---

#### Règle n°1 : l'alerte avant la ressource

Sur un cloud, on n'installe pas la maison avant le détecteur de fumée. **AWS Budgets** :

- Un **budget** = un montant mensuel surveillé (nous : **10 USD/mois**) + des **alertes**.
- Nos trois seuils : e-mail à **50 %** (5 USD — on lève la tête), **80 %** (8 USD — on cherche la fuite), **100 %** (10 USD — on coupe tout et on appelle le formateur).
- On peut alerter sur le coût **réel** (déjà dépensé) ou **prévisionnel** (tendance projetée en fin de mois). Nous prenons le réel, plus simple à raisonner.
- Limite honnête : Budgets **alerte**, il ne **bloque pas**. Les données de facturation ont jusqu'à ~8-24 h de retard. C'est un rétroviseur, pas un frein — le frein, c'est votre discipline de teardown.

💰 Les **2 premiers budgets sont gratuits** sur tout compte. Le nôtre ne coûte rien.

Lien avec le fil conducteur : c'est votre premier réflexe **superviser**.

---



### Démo 1.1 — Compte, MFA root et alerte de budget

~30 min, tous ensemble, chacun sur **SON compte**. Suivez le formateur écran par écran :

1. Connexion **root** (l'e-mail du compte) — la dernière fois ou presque.
2. **MFA sur le root** : application d'authentification sur votre téléphone.
3. **Création du budget** : 10 USD/mois, alertes e-mail à 50/80/100 %. *Avant toute autre chose*.
4. **Activation de l'accès facturation pour IAM** : sinon votre futur utilisateur du quotidien ne verra jamais les coûts.
5. **Tour de la console** : recherche, sélecteur de région, pourquoi une ressource parisienne est invisible depuis Francfort.

📄 Script : [demos/06-cl-aws1/demo-1-1-compte-et-budget.md](#)

⚠ La console évolue : si votre écran diffère de celui du formateur, dites-le — le script prévoit les variantes.

### 3. IAM

#### Identités, groupes, politiques : qui a le droit de faire quoi

#### root vs utilisateurs IAM

Deux types d'identités humaines dans un compte :

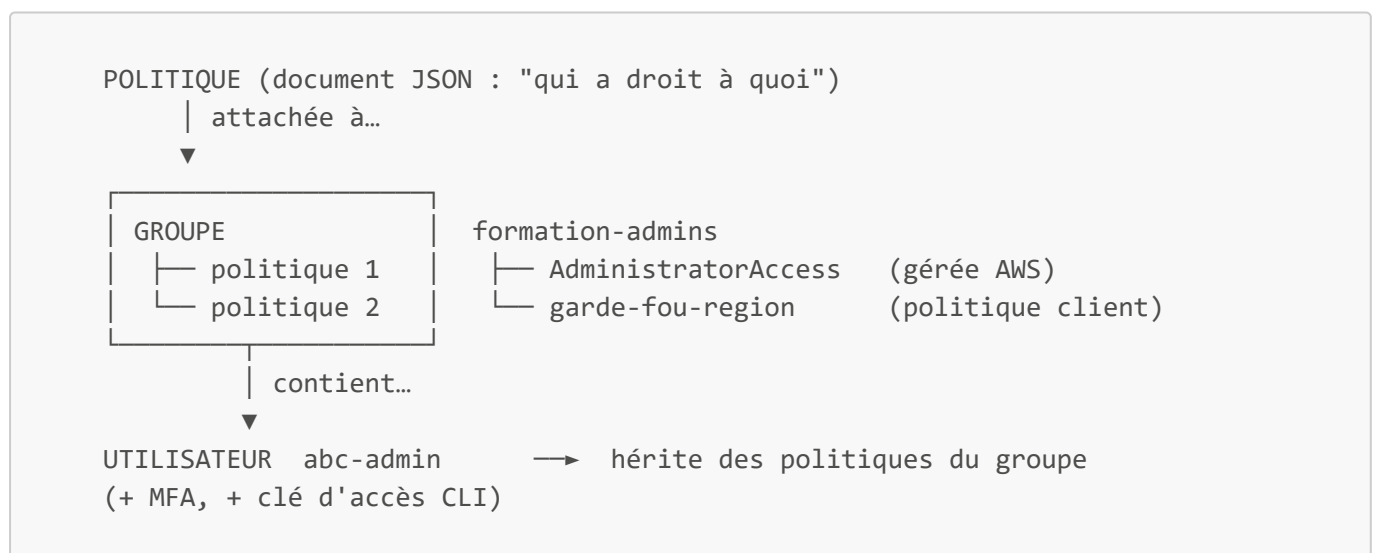
- **Le root** : l'identité créée avec le compte (l'adresse e-mail). Il peut **tout** faire, y compris fermer le compte et changer les moyens de paiement. **Impossible à restreindre** par une politique.
- **Les utilisateurs IAM** : des identités créées *dans* le compte, qui n'ont **aucun droit par défaut** — on leur en donne via des politiques.

La règle professionnelle, non négociable :

1. Jour 1 : MFA sur le root, création d'un utilisateur IAM administrateur.
2. Ensuite : **le root ne sert plus jamais** (hors ~5 tâches exclusives : clôture du compte, changement de plan de support...).
3. **Jamais, jamais de clés d'accès sur le root.**

Analogie Linux (bloc 03) : le root AWS, c'est **root** sur votre serveur — vous avez appris à travailler avec un utilisateur + **sudo**. Même hygiène ici.

#### Utilisateurs, groupes, politiques



Bonne pratique : les politiques s'attachent aux **groupes**, les utilisateurs rejoignent les groupes. On ne colle pas de droits directement sur les personnes — sinon, au 15<sup>e</sup> utilisateur, plus personne ne sait qui a quoi.

#### Anatomie d'une politique JSON (1/2) : le squelette

Une politique = un document JSON qui répond à : *qui a le droit de faire quoi, sur quoi, à quelles conditions ?*

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "LectureSeuleS3",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::mon-bucket/*"
    }
  ]
}
```

- **Version** : la version du *langage* des politiques. Toujours **2012-10-17** (ce n'est pas la date d'écriture !).
- **Statement** : une **liste** de règles — chacune est évaluée indépendamment.
- **Sid** : un identifiant lisible, optionnel — un commentaire structuré.
- **Effect** : **Allow** ou **Deny**. C'est tout. Pas de « peut-être ».

## Anatomie d'une politique JSON (2/2) : Action, Resource, Condition

- **Action** : *quoi* — au format **service:Operation** (**s3:GetObject**, **ec2:RunInstances**, **iam:CreateUser**). Le joker **\*** existe : **s3:\*** = tout S3, **"Action": "\*" = tout**.
- **Resource** : *sur quoi* — un **ARN** (Amazon Resource Name), la « carte d'identité » de chaque ressource :

```
arn:aws:iam::123456789012:user/abc-admin
arn:aws:s3:::formation-documents/confidentiel/plan.pdf
|   |   |   |   |   |   |
arn aws service compte ressource (le chemin)
      (région pour les services régionaux)
```

- **Condition** : *à quelles conditions* — des tests optionnels sur le contexte de la requête :  
**aws:RequestedRegion** (dans quelle région ?), **aws:MultiFactorAuthPresent** (MFA actif ?),  
**ec2:InstanceType** (quel type de machine ?)...

Vous savez lire ces trois champs → vous savez lire **n'importe quelle** politique AWS.

## La logique d'évaluation : le deny gagne toujours

Requête : abc-admin veut faire s3:GetObject sur .../confidentiel/plan.pdf





Deux règles à graver : **tout est interdit par défaut** (deny implicite), et **un Deny explicite bat n'importe quel Allow**, d'où qu'il vienne. C'est exactement le mécanisme de notre garde-fou région.

## Politiques gérées AWS vs politiques client

Trois provenances de politiques, du plus clé-en-main au plus sur-mesure :

- **Gérées AWS** (*AWS managed*) : écrites et maintenues par AWS, prêtes à l'emploi — `AdministratorAccess`, `ReadOnlyAccess`, `AmazonS3ReadOnlyAccess`... Pratiques, mais souvent **plus larges que votre besoin**.
- **Gérées par le client** (*customer managed*) : écrites par **vous**, réutilisables, versionnées. C'est le cas de notre garde-fou région. Le standard en entreprise.
- **Inline** : collées directement *dans* un seul utilisateur/groupe/rôle. Ni réutilisables, ni visibles dans la liste des politiques. À éviter, sauf cas très ponctuel.

Aujourd'hui, pour la formation : `AdministratorAccess` (gérée AWS, pour ne pas être bloqués) + notre garde-fou région (client, en Deny). Le Deny gagnant toujours, l'admin reste enfermé à Paris. En entreprise réelle, on donnerait bien moins qu'admin.

## Notre garde-fou : tout refuser hors de Paris

La politique client que nous attacherons au groupe (fichier `code/06-cl-aws1/politiques/policy-region-eu-west-3-seulement.json`) :

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "RefuserToutHorsParis",
      "Effect": "Deny",
      "NotAction": [ "iam:*", "sts:*", "budgets:*", "ce:*",
                    "support:*", "s3:ListAllMyBuckets" ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": { "aws:RequestedRegion": "eu-west-3" }
      }
    }
  ]
}
```

Lecture : **refuser** (Deny) **tout sauf** (NotAction) les services globaux listés, **partout** (Resource : \*), **si** la région demandée **n'est pas** Paris. Double négation = « hors de Paris, seuls IAM, STS, Budgets... restent permis ». On

la décortique en démo.

---

## MFA : la deuxième serrure

Le **MFA** (Multi-Factor Authentication) exige, en plus du mot de passe, un code à 6 chiffres renouvelé toutes les 30 secondes :

- **Ce que vous savez** (mot de passe) + **ce que vous avez** (votre téléphone).
- Un mot de passe volé — hameçonnage, fuite, réutilisation — ne suffit plus pour entrer.
- Support : application d'authentification (Google Authenticator, Microsoft Authenticator, FreeOTP...), clé physique FIDO, ou jeton matériel.

Sur AWS, le MFA se met **par identité** : aujourd'hui, sur votre **root** (démo 1.1) puis sur votre **utilisateur IAM** (démo 1.2).

Pourquoi c'est si critique sur un cloud ? Parce qu'un compte compromis, ce n'est pas « quelqu'un lit mes fichiers » : c'est quelqu'un qui **crée 200 machines de minage à vos frais** en dix minutes. Le MFA est votre première mesure **sécuriser**.

---

## Bonnes pratiques IAM — la checklist du jour 1

Ce que fait tout administrateur cloud sérieux, dès le premier jour :

- ☒ **MFA sur le root**, puis le root retourne au coffre (le mot de passe aussi).
- ☒ **Jamais de clés d'accès sur le root** — il n'y a aucun cas d'usage légitime.
- ☒ Un **utilisateur IAM par personne** (pas de compte partagé : la traçabilité, c'est *qui a fait quoi*).
- ☒ Les droits vont aux **groupes**, les personnes vont dans les groupes.
- ☒ **Moindre privilège** : donner le minimum nécessaire, élargir si besoin — jamais l'inverse. (Notre **AdministratorAccess** de formation est une exception pédagogique, encadrée par le garde-fou.)
- ☒ MFA aussi sur les utilisateurs IAM ; clés d'accès **jamais** dans le code ni dans un dépôt git.

Même philosophie que le bloc Linux : on ne travaille pas en root, on trace, on limite.



### Exercice 1.1 — Lire des politiques IAM comme un pro

**45 min — difficulté 2/5 — papier/éditeur, aucun accès AWS nécessaire.**

Vous recevez **3 politiques JSON réalistes** :

- (a) accès S3 avec un **Deny explicite** sur un préfixe **confidentiel/** ;
- (b) politique EC2 avec **Conditions** sur la région et le type d'instance ;
- (c) extrait de la politique gérée **ReadOnlyAccess** face à une politique inline.

Pour chacune : *qui peut faire quoi ? que se passe-t-il dans tel scénario ? pourquoi ?* Bonus : **écrire** votre première politique (chacun ne peut lire que sa propre fiche IAM).



Énoncé : [exercices/06-cl-aws1/exercice-1-1-lecture-politiques-iam.md](#)

Savoir **lire** une politique avant d'en écrire, c'est comme lire du code avant d'en produire : c'est là que se joue votre autonomie.

---

## 4. AWS CLI v2

---

### AWS depuis votre terminal — là où commence l'automatisation

---

#### Installer la CLI v2

La CLI s'installe sur **votre poste** (pas sur AWS !). C'est un simple programme qui envoie des requêtes signées à l'API AWS.

**macOS :**

```
curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
sudo installer -pkg AWSCLIV2.pkg -target /
```

**Linux (x86\_64) :**

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

**Windows :** `msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi` (ou l'installateur MSI téléchargé). Puis, sur tous les systèmes :

```
aws --version
# aws-cli/2.27.0 Python/3.13.2 Linux/6.8.0-51-generic exe/x86_64.ubuntu.24
```

L'important : `aws-cli/2.x` — la **v2**. La v1 traîne encore dans de vieux paquets `apt`.

---

### `aws configure` et les fichiers `~/.aws/`

La CLI s'authentifie avec une **clé d'accès** (créée sur votre utilisateur IAM en démo) :

```
aws configure --profile formation
# AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
# AWS Secret Access Key [None]: wJalrXPnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
# Default region name [None]: eu-west-3
# Default output format [None]: json
```

Cela écrit deux fichiers dans votre home — **exactement l'esprit de** `~/.ssh/config` du bloc Linux :

```
# ~/.aws/credentials (les secrets - à protéger comme une clé privée SSH)
[formation]
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXpNFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY

# ~/.aws/config (les préférences, pas de secret)
[profile formation]
region = eu-west-3
output = json
```

Clé d'accès = identifiant (**AKIA...**, public) + **secret** (affiché UNE seule fois à la création).

---

## Les profils nommés : plusieurs identités, un terminal

Un **profil** = un jeu « clés + région + format » nommé. Comme un **Host** dans `~/.ssh/config` : un alias qui embarque toute la configuration de connexion.

Trois façons de choisir le profil :

```
# 1. Option explicite sur chaque commande (notre standard en formation)
aws sts get-caller-identity --profile formation

# 2. Variable d'environnement pour toute la session shell
export AWS_PROFILE=formation
aws sts get-caller-identity

# 3. Le profil [default] – utilisé si on ne précise rien
```

Pourquoi c'est vital : en entreprise vous jonglerez entre comptes *dev*, *prod*, *client-X*. Le profil explicite évite le drame classique : **détruire en prod en croyant être en dev**. Prenez le réflexe `--profile formation` dès aujourd'hui.

---

## Premières commandes : qui suis-je ?

**LA** commande de vérification, à connaître par cœur — l'équivalent de `whoami` :

```
aws sts get-caller-identity --profile formation
```

```
{
  "UserId": "AIDAJQABLZS4A3QDU576Q",
  "Account": "123456789012",
```

```
"Arn": "arn:aws:iam::123456789012:user/abc-admin"
}
```

Elle confirme trois choses : vos **clés fonctionnent**, vous êtes sur le **bon compte** (les 12 chiffres), sous la **bonne identité** (l'ARN). STS = Security Token Service, le service qui gère les identités en cours.

Réflexe professionnel : `get-caller-identity` **avant toute opération sérieuse**, comme un `whoami` && `hostname` avant un `rm -rf` sur un serveur.

---

## Explorer les régions et les AZ

```
aws ec2 describe-regions --profile formation
```

→ un gros JSON avec toutes les régions actives. Et les AZ de Paris :

```
aws ec2 describe-availability-zones --region eu-west-3 --profile formation
```

```
{
  "AvailabilityZones": [
    { "State": "available", "RegionName": "eu-west-3",
      "ZoneName": "eu-west-3a", "ZoneId": "euw3-az1", "ZoneType":
"availability-zone" },
    { "State": "available", "RegionName": "eu-west-3",
      "ZoneName": "eu-west-3b", "ZoneId": "euw3-az2", "ZoneType":
"availability-zone" },
    { "State": "available", "RegionName": "eu-west-3",
      "ZoneName": "eu-west-3c", "ZoneId": "euw3-az3", "ZoneType":
"availability-zone" }
  ]
}
```

Paris a **3 AZ**. `--region` surcharge ponctuellement la région du profil. Mais ce JSON est verbeux... d'où les deux options qui suivent.

---

### `--query` (filtrer) et `--output` (formater)

`--query` filtre le JSON **côté client** avec le langage **JMESPath** :

```
aws ec2 describe-regions --query "Regions[].RegionName" \
  --output table --profile formation
```

- `Regions[]` : parcourir la liste `Regions` ; `.RegionName` : ne garder que ce champ.
- `length(...)` compte : `--query "length(AvailabilityZones)" → 3`.

`--output` choisit le format : `json` (complet, pour `jq/scripts`), `table` (lisible, pour les humains), `text` (brut, tabulé — idéal pour les boucles bash) :

```
aws ec2 describe-availability-zones --region eu-west-3 \
    --query "AvailabilityZones[].ZoneName" --output text --profile formation
# eu-west-3a    eu-west-3b    eu-west-3c
```

`--output text` + boucle `for` du bloc Linux = vos premiers scripts d'inventaire. C'est le début du **automatiser**.

## Démo 1.2 — IAM en console, puis la CLI

~45 min, en deux temps. Vous ferez la même chose sur votre compte.

**Temps 1 — console IAM** (connecté en root, dernière fois !) :

- groupe `formation-admins` : `AdministratorAccess` + garde-fou région (lu **ligne à ligne**) ;
- utilisateur `$PREFIX-admin` (ex. `abc-admin`) dans le groupe, mot de passe, **MFA** ;
- création de la **clé d'accès** (le secret ne s'affiche qu'une fois !).

**Temps 2 — terminal** :

- installation CLI v2, `aws configure --profile formation` ;
- `aws sts get-caller-identity`, `describe-regions --output table`, `aws iam list-users` ;
- visite de `~/.aws/credentials` et `~/.aws/config`.

 Script : `demos/06-cl-aws1/demo-1-2-iam-cli.md`

À la fin : **plus personne ne se connecte en root**. Plus jamais (ou presque).

## Exercice 1.2 — L'explorateur CLI

**30-40 min — difficulté 2/5 — votre terminal, profil `formation`.**

Sans toucher à la console, uniquement à la CLI :

1. Vérifier votre **identité** (compte, ARN).
2. Lister les **régions**, puis les **AZ** d'`eu-west-3`.
3. **Compter** les AZ avec `--query` (sans compter à la main !).
4. Tableau des **politiques attachées** à votre groupe.
5. Créer un **second profil** nommé et basculer de l'un à l'autre.
6. Produire du `--output text` prêt à scripter.

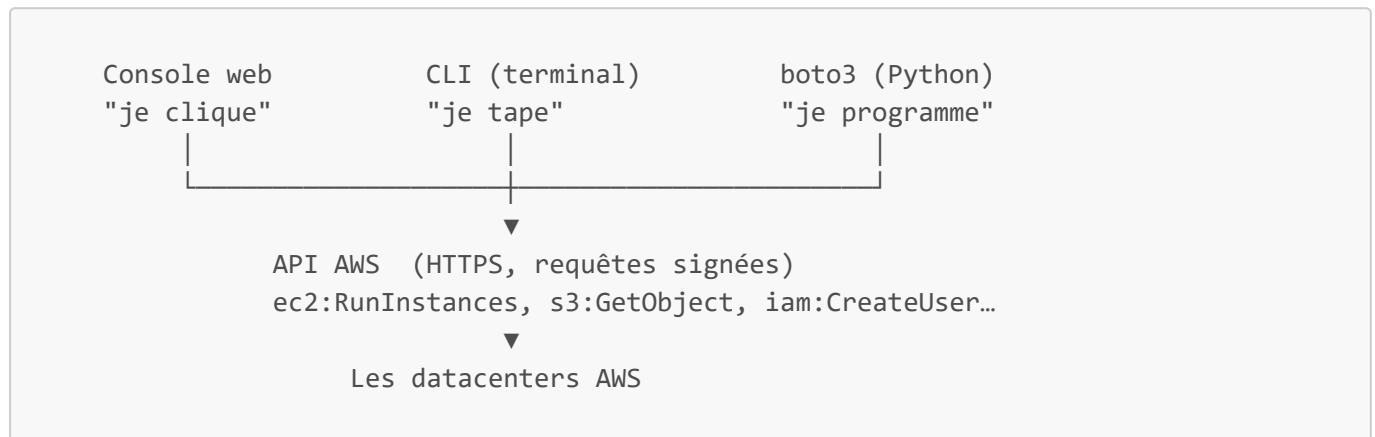
Bonus : une **boucle bash** qui affiche le nombre d'AZ par région européenne — avec une surprise instructive au passage... 

Énoncé : [exercices/06-cl-aws1/exercice-1-2-explorateur-cli.md](#)

## 5. Tout est API

La console n'est qu'un client parmi d'autres

Trois clients, une seule API



**Chaque** action AWS — chaque clic de la console — est un appel d'API HTTPS signé. La console est un client web, la CLI un client shell, **boto3** (le SDK Python — rendez-vous J8) un client programmable, comme **requests** face à l'API StockLine.

Conséquence énorme : **tout ce qui se clique peut se scripter**. C'est le fondement du métier — « automatiser, superviser, sécuriser » — et de l'Infrastructure as Code du bloc 11. La console sert à *découvrir* ; la CLI et le code servent à *travailler*.

### 💰 Combien a coûté cette journée ?

Faisons les comptes, service par service :

| Ce qu'on a utilisé                                                         | Coût                              |
|----------------------------------------------------------------------------|-----------------------------------|
| Compte AWS (exister ne coûte rien)                                         | 0 €                               |
| AWS Budgets — notre alerte 10 USD                                          | 0 € (2 premiers budgets gratuits) |
| IAM — groupes, utilisateurs, politiques, MFA                               | 0 € (toujours gratuit)            |
| AWS CLI v2 — logiciel sur votre poste                                      | 0 €                               |
| Appels d'API en lecture ( <b>describe-*</b> , <b>list-*</b> , <b>sts</b> ) | 0 €                               |

**Total : 0 €** — et ce sera vérifiable dans quelques heures dans Billing.

Retenez la méthode plutôt que le résultat : **avant** d'utiliser un service, on se demande ce qu'il coûte ; **après**, on vérifie. Demain, avec EC2, le compteur commencera vraiment à tourner — et vous saurez le surveiller.

## Récap — les points clés du jour

- Un **compte AWS** = frontière de **facturation** + frontière de **sécurité**. Identifiant à 12 chiffres.
- **Région** (lieu, ex. eu-west-3 = Paris : latence, RGPD, prix)  $\supset$  **AZ** (datacenters indépendants). Les ressources sont **régionales** : mauvaise région = ressources « invisibles ».
- **Budget d'abord** : alerte 10 USD à 50/80/100 % *avant* toute ressource. Budgets alerte, ne bloque pas.
- **root** : MFA puis au coffre. Jamais de clés d'accès root. Le quotidien = **utilisateur IAM** dans un **groupe** porteur des politiques.
- Politique JSON : **Effect** / **Action** / **Resource** / **Condition**. **Deny par défaut**, et **Deny explicite** > **tout Allow** (notre garde-fou région).
- **CLI v2** : `aws configure --profile formation`, secrets dans `~/.aws/credentials`, `sts get-caller-identity = whoami`, `--query (JMESPath)` + `--output json|table|text`.
- **Tout est API** : console, CLI, boto3 = trois clients d'une même API → tout est automatisable.

---

## Quiz de fin de journée

---

### 10 questions — réponses en débriefing avec le formateur

---

#### Quiz — questions 1 et 2

**Question 1.** Un compte AWS constitue une frontière de... ?

- A. Facturation uniquement
- B. Sécurité uniquement
- C. Facturation **et** sécurité
- D. Ni l'un ni l'autre : c'est la région qui isole

**Question 2.** Vous avez créé une ressource dans eu-west-3 (Paris). Votre console est positionnée sur eu-central-1 (Francfort). Que voyez-vous ?

- A. La ressource, avec une mention « autre région »
- B. Rien : la plupart des ressources sont régionales
- C. La ressource, car la console affiche tout le compte
- D. Une erreur de connexion

---

#### Quiz — questions 3 et 4

**Question 3.** Pourquoi avons-nous créé l'alerte de budget **avant** toute autre ressource ?

- A. AWS l'exige pour activer le compte
- B. Pour être prévenus par e-mail avant qu'un oubli ne devienne une facture
- C. Parce que Budgets bloque automatiquement les dépenses à 100 %
- D. Pour débloquer le free-tier

**Question 4.** Une identité a une politique qui **autorise** `s3:GetObject` sur tout un bucket, et une autre politique qui **refuse explicitement** (**Deny**) `s3:GetObject` sur `confidentiel/*`. Elle demande un objet dans

confidentiel/. Résultat ?

- A. Autorisé : l'Allow couvre tout le bucket
  - B. Refusé : un Deny explicite l'emporte sur tout Allow
  - C. Autorisé si elle a le MFA
  - D. Cela dépend de l'ordre des politiques dans le JSON
- 

## Quiz — questions 5 et 6

**Question 5.** Aucune politique attachée à un utilisateur ne mentionne l'action `ec2:RunInstances` (ni Allow, ni Deny). Il tente de lancer une instance. Résultat ?

- A. Autorisé : rien ne l'interdit
- B. Refusé : sans Allow explicite, tout est interdit (deny implicite)
- C. Autorisé, mais avec un avertissement
- D. AWS demande une confirmation au root

**Question 6.** Quel élément d'une politique permet de restreindre les actions à la seule région eu-west-3 ?

- A. `"Resource": "eu-west-3"`
  - B. `"Sid": "eu-west-3"`
  - C. Une `Condition` sur la clé `aws:RequestedRegion`
  - D. Le champ `Version`
- 

## Quiz — questions 7 et 8

**Question 7.** Après la configuration initiale du compte (MFA, création de l'admin IAM), à quoi sert encore le root au quotidien ?

- A. À se connecter chaque matin, c'est le compte principal
- B. À lancer les commandes CLI
- C. À rien, ou presque : seules quelques tâches exclusives le nécessitent
- D. À créer les ressources EC2

**Question 8.** Quelle commande permet de vérifier avec quelle identité et sur quel compte la CLI travaille ?

- A. `aws iam whoami`
  - B. `aws sts get-caller-identity`
  - C. `aws configure list-profiles`
  - D. `aws ec2 describe-identity`
- 

## Quiz — questions 9 et 10

**Question 9.** Où la CLI stocke-t-elle la clé d'accès secrète du profil `formation` ?

- A. Dans `~/.aws/config`
- B. Dans `~/.aws/credentials`
- C. Dans une variable système chiffrée par AWS

- D. Sur les serveurs AWS, jamais localement

**Question 10.** Console web, AWS CLI et boto3 ont un point commun fondamental. Lequel ?

- A. Ils nécessitent tous un navigateur
  - B. Ce sont trois clients de la même API AWS
  - C. Ils sont tous les trois payants à l'usage
  - D. Ils ne fonctionnent que depuis le root
- 

## À demain !

Aujourd'hui, vous avez fait ce que 90 % des débutants ne font pas : **sécuriser et surveiller le compte avant de créer quoi que ce soit**. Votre sandbox est prête, budgétée, verrouillée sur Paris, pilotable au terminal.

### Demain — Jour 2 : EC2, vos premières machines virtuelles 🚀

- Les **AMI** : des images de machines prêtes à démarrer (votre Ubuntu du bloc 03... en 40 secondes).
- Les **types d'instances** : t3.micro et ses cousines — choisir la bonne taille (et rester dans le free-tier).
- Les **key pairs** : le retour de vos clés SSH.
- Le **user data** : un script qui configure la machine au premier démarrage — StockLine déployée *sans se connecter à la machine*. L'automatisation commence.

D'ici là : vérifiez que vous avez bien reçu l'e-mail de confirmation d'AWS Budgets, et que `aws sts get-caller-identity --profile formation` répond. À demain !